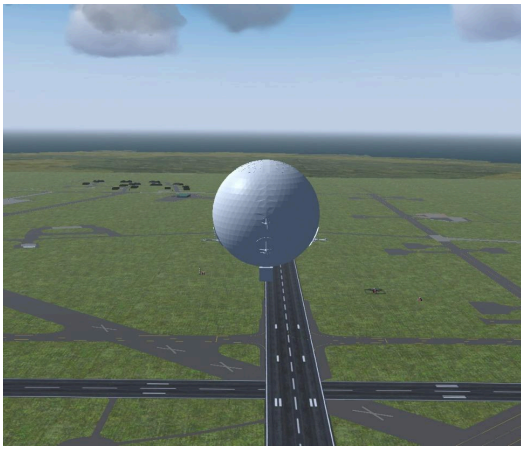
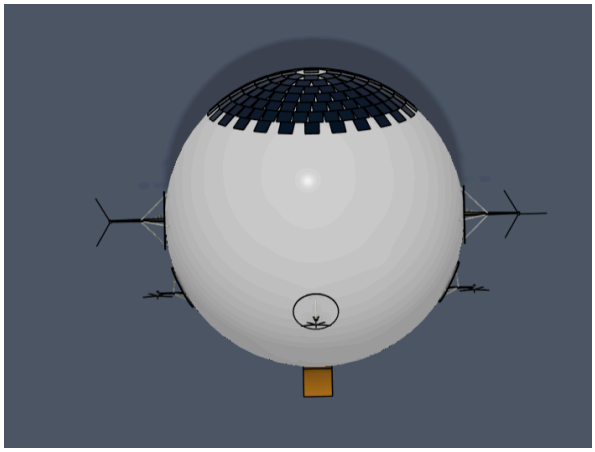



Description

This application was developed for the Knowledge-Based-Engineering course in collaboration with Aerolympics, focused on the conceptual design and optimization of a solar-powered spherical airship to facilitate the transport of medical supplies in Africa from Points of Care to Points of Need. This project involves developing a python based (object-oriented) system that integrates parametric geometry generation, automated CAD output, and simulation workflows, enabling rapid optimization of design configurations based on the mission profile. The application manages tightly coupled relationships between geometry, buoyancy, propulsion, and energy systems, while automating outputs compatible for simulation environments (JSBSim/FlightGear).

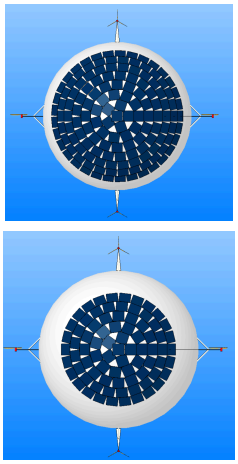
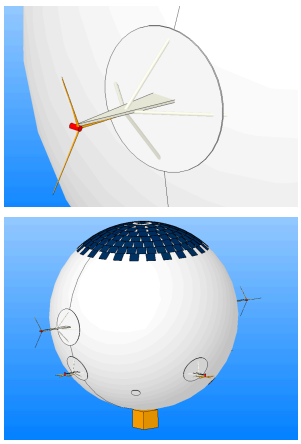
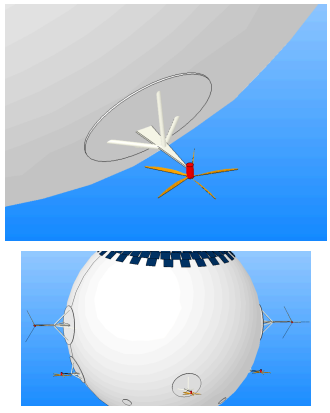
The first interface between the user and the KBE app is the webpage developed, called 'spherical_airship.html'. This is the primary interface responsible for generating the input file for the KBE application for both the forward and inverse design mode. Depending on the mode of operation, the required parameters will be displayed on the screen with their appropriate units which the user has to fill out. After doing so, the user can proceed to generate the Input file for the KBE application, following which they can run KBE_airship python file which reads the input file and based on the mode performs the required calculations. The user can then generate a STEP file or a XML file through ParaPy, and even generate a PDF with all the inputs and a summary of calculations.

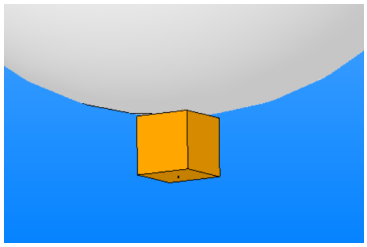
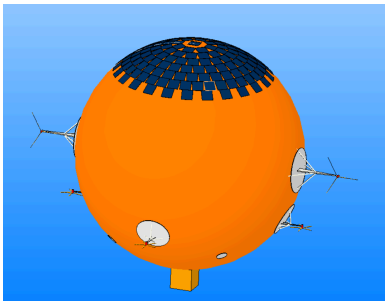
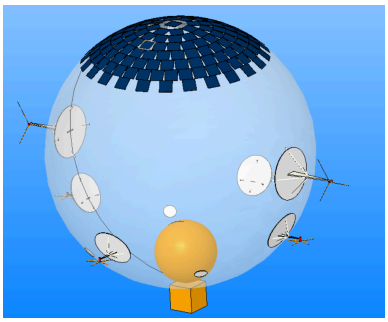
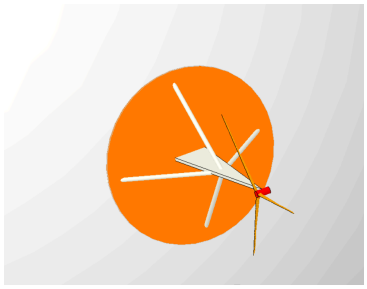
Note!

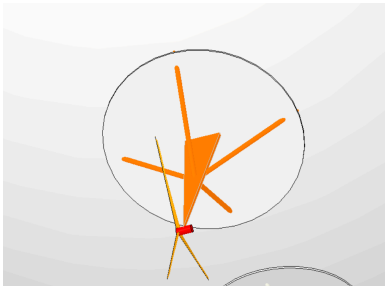
MATLAB Engine API for Python has to be installed along with ParaPy.

Assembly

The assembly of the entire airship has the following components:

Component /Class name	Controlling Parameters and design variables	Description	Picture
SolarPanel	CP Energy requirement; Available solar power; DV Area per panel; Weight per panel; Required number of panels; Location	Component responsible for solar panel layout on the airship.	
HorizontalPropulsion	CP Required thrust based on average wind speed (gives airspeed) and drag; Thrust requirement; DV Propeller radius; Number of blades; Chord and twist spanwise distribution; Location	Component responsible for handling the propulsion architecture in the horizontal direction for movement.	
VerticalPropulsion	CP Required high thrust vs rpm slope for high stability; DV Propeller radius; Number of blades; Chord and twist spanwise distribution; Location;	Component responsible for handling the propulsion architecture in the vertical direction for stability.	

Battery	CP Energy requirement; DV Dimensions; Weight per battery; Energy capacity per battery	Component responsible for storing energy in situations where there is high cloud cover and solar panels are not enough. Also the primary energy source during takeoff and landing phase of the airship.	
Payload	CP Given by user DV Dimensions; Weight;	Component responsible for the payload which is defined by the user.	
Envelope	CP Number of solar panels; Lift requirement; Cruise altitude; DV Envelope mass per m2; Envelope radius; Envelope thickness;	Component responsible for modelling the outer envelope of the spherical airship which contains the ballonet and the lifting gas.	
Ballonet	CP Lift requirement; Cruise Altitude; DV Ballonet radius; Ballonet thickness; Ballonet mass per m2; Ballonet volume fraction;	Component inside the outer envelope which holds the ambient gas inside.	
LoadPatchHorizontal/ LoadPatchVertical/ SinglePayloadPatch	CP Propeller and strut dimensions; Location on airship; DV Location; Thickness;	Component responsible for modelling the attachment points of the support structures of the propulsion system and the payload with the envelope.	

	Density of material; Dimensions;		
StrutSupport/ StrutSupporH	CP Propeller dimensions; DV Strut size; Additional supports dimension; Orientation; Strut radius;	Components responsible for the support structure for the propulsion systems on the aircraft.	

Paths

All the paths required for enabling the interactions between the modules and the other code files are listed in the file called 'env_Arnesh' and is the only file in which the paths need to be updated.

Modes of Operation

User has the option to work with two different modes:

1. Forward Mode
2. Inverse Mode

1. Forward Mode

Description

Forward mode is to be utilized when the geometry is given by the user to the application. No analysis is conducted and this functionality is provided for situations where the user has a specific design that they want to test.

Inputs

Input	Characteristics	Location
Envelope		
Envelope radius	Units: m	Webpage
Envelope thickness	Units: m	Webpage
Envelope mass per m2	Units: kg/m^2	Webpage
Gas Density (lifting)	Units: kg/m^3	Webpage
Gas mass (lifting)	Units: kg	Webpage
Ballonet		
Ballonet radius	Units: m	Webpage
Ballonet thickness	Units: m	Webpage
Ballonet mass per m2	Units: kg/m^2	Webpage
Solar Panels		
Required solar panel area	Units: m^2	Webpage
Solar panel area per panel	Units: m^2	Webpage
Weight per panel	Units: kg	Webpage
Batteries		
Number of batteries	Units: -	Webpage
Energy capacity per battery	Units: J	Webpage
Battery density	Units: kg/m^3	Webpage

Battery dimensions	Units: m (3)	Webpage
Vertical Propulsion		
Number of blades	Units: -	Webpage
Propeller radius	Units: m	Webpage
Location	Units: -	Webpage
Strut size	Units: m	Webpage
Strut radius	Units: m	Webpage
Spanwise distribution of normalized chord and twist	Units: -	Excel file: blade_data_output_vertical Path: Main directory
Horizontal Propulsion		
Number of blades	Units: -	Webpage
Propeller radius	Units: m	Webpage
Location	Units: -	Webpage
Strut size	Units: m	Webpage
Strut radius	Units: m	Webpage
Spanwise distribution of normalized chord and twist	Units: -	Excel file: blade_data_output2 Path: Main directory
Payload		
Payload Weight	Units: kg	Webpage
Payload d factor	Units: -	Webpage
Payload dimensions	Units: m (3)	Webpage
Load Patches		
Load patch radius (payload)	Units: m	Webpage

Location	Units: -	Webpage
Thickness	Units: m	Webpage
Density	Units: kg/m^3	Webpage

Instructions

Step 1: Fill the spherical_airship web application (forward mode) with the required inputs.
Step 2: Generate the required input file.
Step 3: Make sure the propeller geometry data is in the appropriate directory (horizontal and vertical thrusters) with the given naming convention.
Step 4: Run KBE_airship
Step 5: Analyze geometry and ensure there are no warnings.
Step 6: Generate the pdf. Also, generate the .ac file. Then create the .xml file. After this, your spherical airship can be controlled in FlightGear. Basic controls for FlightGear include horizontal movement which is controlled using the A and D keys. Pressing A increases the port (left) engine thrust forward, while lowercase a reverses it. D and d do the same for the starboard (right) engine. There is no rudder, and yawing left or right is achieved by running the two horizontal engines at different power levels. Pressing C or c at any time immediately zeroes all engines as an emergency stop.

2. Inverse Mode

Description

Forward mode is to be utilized when the geometry is given by the user to the application.

Modules/Packages Requirements

Index	Python module name	Dependencies	Description
1	airship_trajectory	`pandas` `geographiclib`	This script generates a simulated airship trajectory between geographic waypoints loaded from an Excel file. It computes accurate distances and interpolated positions along the Earth's surface while assuming a constant ground speed. The model produces time-stepped trajectory data including latitude, longitude, elapsed time, cumulative distance traveled, and UTC timestamps.
2	envelope_buyoancy_module		
3	envelope_drag	`ambiance`	This module computes the aerodynamic drag force acting on a spherical airship envelope. The drag coefficient is evaluated using an extended piecewise sphere correlation based on the work of Clift et al., providing drag predictions across all major Reynolds number regimes, from creeping flow to highly turbulent flow. Atmospheric properties are obtained from the ICAO Standard Atmosphere model using Ambiance.
4	ground_speed	`geographiclib`	Computes airship ground speed and total mission distance from a sequence of geodesic waypoints read directly from Excel.
5	horizontal_thruster_module		This module computes the electrical power consumption and energy usage of the two horizontal thrusters on a spherical airship. The aerodynamic propeller performance is precomputed in MATLAB using `AnalyzeProp`, which provides the shaft power required per thruster while already accounting for propeller efficiency effects. The module then applies the motor efficiency to convert shaft power into electrical power demand and integrates this over the timestep duration to determine the total electrical energy consumed.
6	propeller_weight		This module estimates the structural mass of a propeller blade set by integrating the blade volume derived from the chord and twist

			distributions exported by `DesignProp_updated`. The blade is modeled as a solid structure, with the airfoil cross-sectional area approximated from the local thickness-to-chord ratio using a simplified NACA-like airfoil area relation. Chord lengths along the span are reconstructed from the normalized chord distribution and the maximum chord value, while the blade span extends from the hub radius to the blade tip. The total blade volume is obtained through trapezoidal integration across the radial stations and converted to structural weight using a specified material density, with CFRP used as the default material assumption.
7	solar_panel_min_radius		Module that computes the minimum envelope radius such that all solar panels fit above the equator (polar angle < $\pi/2$). Uses the same ring-layout algorithm as SolarPanel._ring_layout in the ParaPy assembly, so results are consistent with the CAD model.
8	solar_power_generated	`pandas` `PySolar`	This module estimates the solar energy generation capability of the airship. It computes the instantaneous solar power production throughout a mission by evaluating the sun position, panel orientation, atmospheric solar irradiance, and panel efficiency at discrete time steps along the flight path. The model determines which panels are illuminated based on the airship heading and solar incidence angle, and integrates the resulting electrical power over time to calculate the total mission energy yield.
9	sun_model	`PySolar`	This module computes the solar incidence angle on a specific panel mounted to the surface of a spherical airship. Using solar position calculations from PySolar, it determines the sun direction vector based on the airship's geographic position, altitude, heading, and UTC time, then transforms the sun vector into the airship body reference frame. The model compares the incoming sunlight direction with the panel's outward-facing normal vector to evaluate the incidence angle, determine whether the panel is illuminated, and estimate the relative solar exposure conditions.
10	time_elapsed_module		Adds an elapsed time to a given start

			date/time and returns the new date/time.
11	wind_data	`pandas` `openpyxl`	This module performs a grid-based wind lookup using an Excel dataset to return local wind conditions at a given latitude and longitude.
12	wind_vector_module		This module computes the full velocity vector decomposition between an airship and the surrounding wind field in a North–East reference frame. It takes wind speed and direction (blowing-to convention) along with the airship's ground speed and heading, then converts both into vector components to determine the relative airspeed experienced by the vehicle. From this, it calculates the airspeed magnitude, as well as the North and East components required for downstream aerodynamic drag and thruster control modules. It also derives the thrust bearing and crab angle, enabling the airship to compensate for crosswinds and maintain a desired heading.

Index	MATLAB module name	Dependencies	Description
1	run_airship_analysis	**Mapping Toolbox**	To compute the heading of the airship given the starting and destination coordinates.

Inputs

Input	Characteristics	Location
Mission timing		
Start Date	dd-mm-yyyy	Webpage
Start Time	Select timezone from dropdown	Webpage
Destination Date	dd-mm-yyyy	Webpage
Destination Reached Time	Select timezone from dropdown	Webpage
Flight Conditions		
Cruise altitude	Units: m	Webpage
Ceiling Tolerance Factor		Webpage
Loiter Coefficient		Webpage
Global Timestep	Units: min	Webpage
Envelope		
Envelope thickness	Units: m	Webpage
Envelope mass per m2	Units: kg/m^2	Webpage
Lifting gas		
Gas type	Drop-down menu	Webpage
Gas density	Units: kg/m^3	Webpage
Ballonet		
Ballonet Max Volume Fraction	Units: -	Webpage
Ballonet Thickness	Units: m	Webpage
Ballonet mass per m2	Units: kg/m^2	Webpage
Energy System		
Energy Capacity per	Units: J	Webpage

Battery		
Solar panel efficiency	Units: -	Webpage
Motor efficiency	Units: -	Webpage
Payload		
Payload Weight	Units: kg	Webpage
Payload d factor	Units: -	Webpage
Payload dimensions	Units: m (3)	Webpage
Load Patches		
Load patch radius (payload)	Units: m	Webpage
Location	Units: -	Webpage
Thickness	Units: m	Webpage
Density	Units: kg/m^3	Webpage

Furthermore, the mission planner has to be filled out with the waypoints and the corresponding wind data.

Instructions

Step 1: Fill the spherical_airship web application (inverse mode) with the required mission profile and generate the config file.
Step 2: Click on the Mission Planner button at the end of the webpage. Fill the sheet based on the mission requirements. User needs to first fill the waypoints page and then the wind page. Finally, generate the second excel file.
Step 3: Run KBE_airship with the correct paths in the env_Arnesh.py script. Note: This may take a while.
Step 4: Analyze generated geometry and ensure there are no warnings.
Step 5: Generate the pdf. Also, generate the .ac file. Then create the .xml file. After this, your spherical airship can be controlled in FlightGear. Basic controls for FlightGear include horizontal movement which is controlled using the A and D keys. Pressing A increases the port (left) engine thrust forward, while lowercase a reverses it. D and d do the same for the starboard (right) engine. There is no rudder, and yawing left or right is achieved by running the two horizontal engines at different power levels. Pressing C or c at any time immediately zeroes all engines as an emergency stop. Vertical movement uses W to increase upward thrust and S to increase

downward thrust, each stepping in 5% increments per keypress. Pressing Space instantly zeroes the vertical thrust command without affecting the horizontal engines.

Implemented Warnings

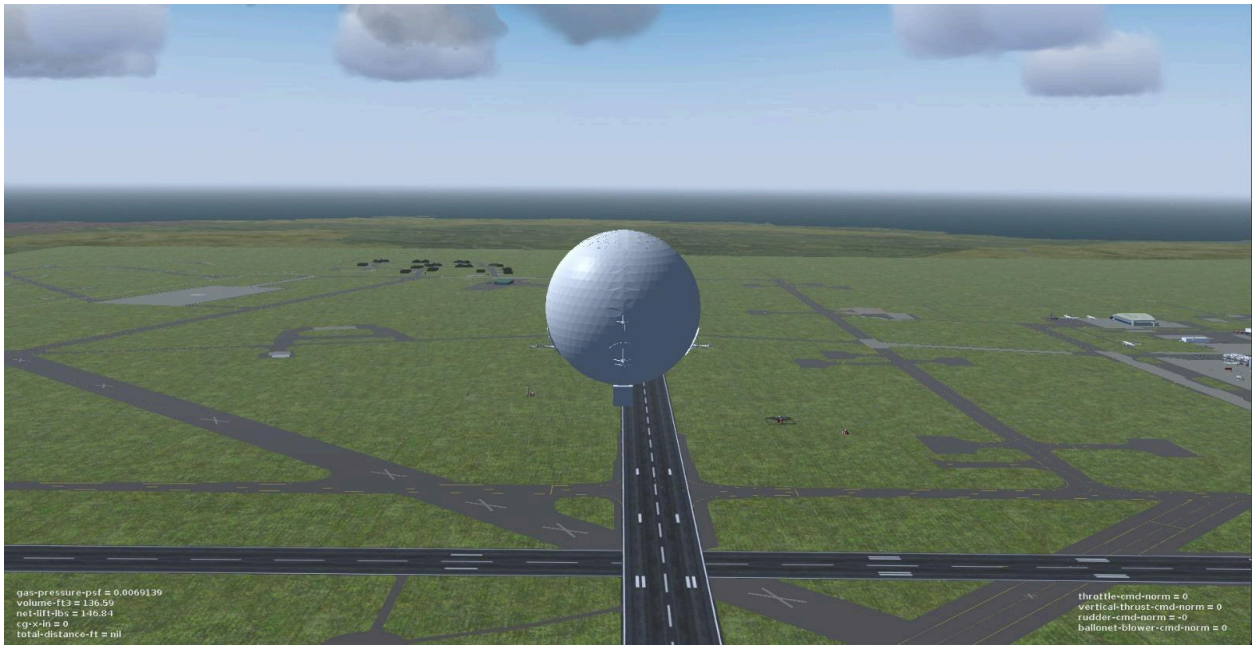
The minimum envelope radius check ensures the sphere is always large enough to tile the required solar panel area above the equator. The payload d-factor validation enforces a value greater than 1, so as to prevent the payload geometry from intersecting the envelope. The strut size check confirms that each strut is long enough to clear its attached propeller radius.

Warnings not implemented due to time constraints

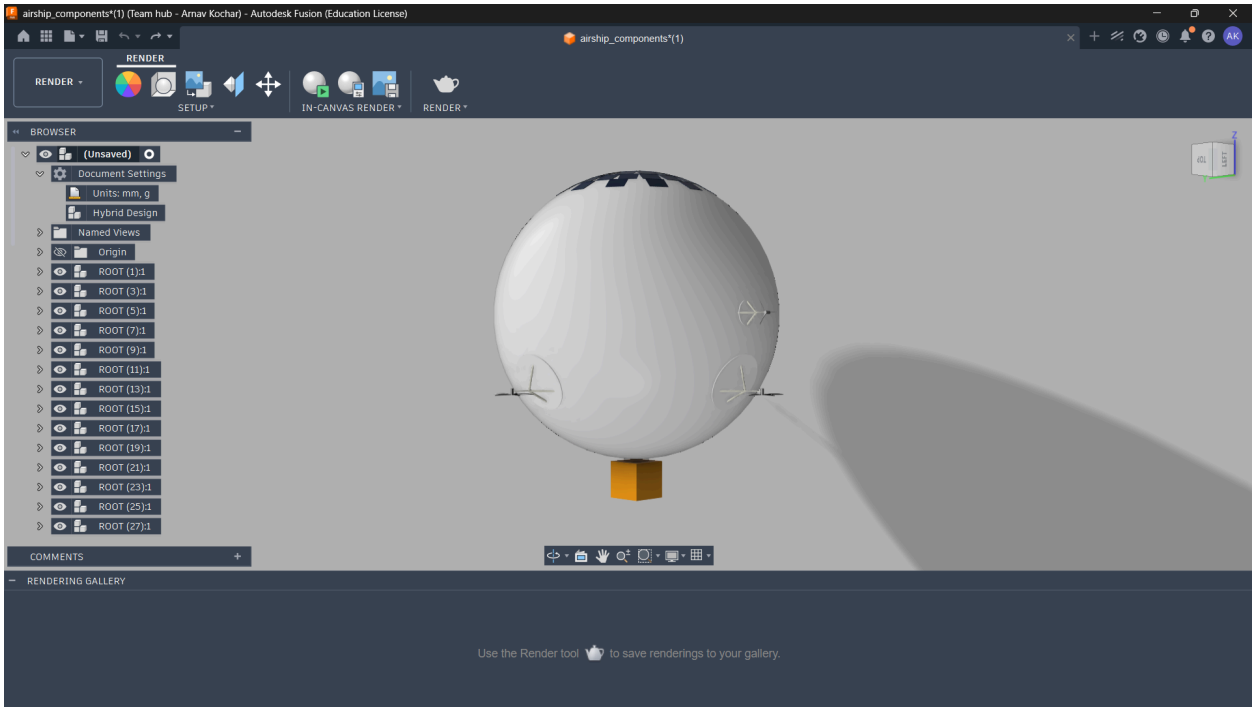
A buoyancy check should verify that the total helium lift at the specified fullness and altitude is sufficient to carry the combined structural, payload, battery, and solar mass and should flag designs where the airship cannot achieve positive net lift. The payload capacity warning would catch unrealistic loading scenarios by computing the maximum payload a given envelope radius can support and warning when the specified payload weight exceeds that. Finally, a propulsor location conflict check should detect cases where the horizontal and vertical propulsor mounting positions resolve to the same point on the envelope surface, which would cause geometry clashes.

Outputs

1. **PDF report**
2. **Inverse mode - Optimized geometry file for chord and twist spanwise distribution**
3. **Xml and FlightGear Implementation:**



4. **Step File Generation:**



Conclusion and Recommendations

In this study, a KBE application for the optimization and analysis of a spherical airship intended for humanitarian aid missions has been developed. A simulation-based optimization framework was implemented to model the energy generation and consumption of the solar panels and propulsion system as functions of time and geographic location. Due to time constraints, several simplifications and assumptions were introduced in various components and performance analysis modules. These aspects should be further improved to increase model fidelity and predictive accuracy. Nevertheless, the framework was designed with modularity in mind, allowing additional components, functionalities, and analyses to be incorporated with minimal modifications.

One of the key simplifications is that only the cruise phase of the mission was explicitly modeled. Takeoff and landing phases, including the deployment of vertical thrusters for stability under wind gust disturbances, were indirectly represented through a loiter coefficient. A more realistic approach would involve explicitly modeling these flight phases. For example, stochastic behavior could be introduced, where gust-induced vertical thruster activation occurs probabilistically at each timestep (e.g., a 1/6 probability), enabling a more representative assessment of operational conditions.

Additionally, the propeller geometry was determined using optimized values for thrust and RPM, which were treated as design variables. A more comprehensive approach would instead define

geometric parameters as the primary design variables, allowing a broader design space to be explored and potentially yielding more optimal propulsion configurations.

The solar energy generation model could also be improved by accounting for environmental effects such as weather conditions, cloud cover, temperature-dependent efficiency degradation, and dust accumulation on solar panels. Furthermore, payload support structures were not modeled in the current framework. The inclusion of flexible support structures would likely influence the dynamic behavior and stability characteristics of the spherical airship during flight and should therefore be considered in future developments.

Finally, several warnings and validation checks were identified as necessary to ensure that the user inputs provided in forward mode remain physically realistic and consistent. However, due to time constraints, only a limited number of these validation features could be implemented. Future work should focus on expanding these checks to improve robustness, prevent unphysical configurations, and enhance the overall reliability of the application.